

Mapa de Características para o Perceptron

Explicação detalhada da submissão — Desafio 1
GBC073 — Inteligência Computacional (FACOM/UFU)

Douglas Ferreira Martins
Gullit Damião Teixeira de Campos

Arquivo de referência: `desafio1_aluno.py`
Classe documentada: `Submissao`

Escore obtido: S = 105.9 / 125

Este documento explica, passo a passo, cada decisão de projeto do mapa de características `phi(X)` entregue e a justificativa matemática de cada termo.

1. Visão geral e filosofia de projeto

O Perceptron treinado pelo harness é um classificador estritamente **linear**: ele só consegue separar duas classes com um hiperplano. As tarefas de teste (`xor` , `duas_luas` , `circulos` , `espiral` , `esfera_10d`) não são linearmente separáveis em suas coordenadas originais — cada uma tem uma fronteira de decisão curva, radial, ou do tipo "produto" (como o XOR).

A função `phi(X)` tem uma única responsabilidade: projetar os dados de `d` dimensões originais para um espaço de até 64 dimensões onde essas fronteiras curvas **se tornem aproximadamente lineares**. O Perceptron então resolve o problema já transformado.

Princípio central adotado: cada termo incluído no mapa precisa ter uma justificativa que valha para *qualquer* tarefa da mesma família — não apenas para as 5 tarefas visíveis usadas para validar o código. Por isso, nenhuma constante numérica foi escolhida observando o resultado em uma tarefa específica (nada de "esse valor funcionou bem para a espiral"); toda constante usada vem de um princípio geral (aritmética de orçamento, propriedade de álgebra linear, ou calibração pelos próprios dados de treino da tarefa em questão — nunca pelo rótulo, e nunca por tentativa e erro olhando o placar).

O mapa final combina quatro blocos de características, cada um resolvendo uma classe diferente de fronteira de decisão:

Bloco	Custo (dimensões)	Tipo de fronteira que resolve
Padronizado (X_5)	<code>d</code>	Já linear na entrada (base sempre incluída)
Raio² ($\sum x_i^2$)	1 (condicional)	Radialmente simétrica (ex.: esfera, círculo)
Quadrados individuais (x_i^2)	até <code>d</code>	Elíptica alinhada aos eixos
Pares cruzados ($x_i x_j$)	até <code>C(d,2)</code>	Tipo produto / rotacionada (ex.: XOR)
Fourier aleatórias ortogonais (RFF/ORF)	o restante do orçamento	Qualquer fronteira suave (aproximação de kernel RBF)

2. Passo a passo do `fit(X)`

O método `fit` recebe apenas as entradas de treino (sem rótulos, como exige o enunciado) e prepara todos os parâmetros que `phi` vai usar depois: médias, desvios-padrão, quais blocos de características estarão ativos, e as matrizes aleatórias (mas determinísticas) usadas pelas features de Fourier.

2.1 Padronização

```
self.mu = X.mean(0)
self.sd = X.std(0) + 1e-8
Xs = (X - self.mu) / self.sd
```

Cada coluna é recentralizada (média 0) e reescalada (desvio-padrão 1). Isso é essencial por dois motivos: (i) as features de Fourier abaixo dependem de uma escala de distância consistente entre as colunas — sem padronizar, uma coluna com valores na casa de milhares dominaria a projeção; (ii) o termo `1e-8` evita divisão por zero caso alguma coluna do treino tenha desvio-padrão nulo (ex.: uma característica constante).

2.2 Orçamento de dimensões

```
rem = self.DIM_MAX - d
reserva_rff = max(rem // 4, 4)
orcamento = max(rem - reserva_rff, 0)
usado = 0
```

`rem` é quanto sobra do limite de 64 depois de reservar as `d` dimensões padronizadas. Uma **reserva mínima** (pelo menos 1/4 do que sobra, e nunca menos que 4 dimensões) é separada só para as features de Fourier *antes* de qualquer termo polinomial disputar esse espaço — isso garante que o mapa nunca vire "só polinomial": mesmo que os termos quadráticos/cruzados coubessem inteiros, ainda sobra capacidade para aproximar fronteiras que não sejam quadráticas nem tipo-produto. `orcamento` é o que resta disponível para os termos estruturados (quadrados, raio, pares).

2.3 Quadrados individuais

```
squares_dim = min(d, orcamento)
orcamento -= squares_dim; usado += squares_dim
self.squares_dim = squares_dim
```

Inclui x_i^2 para tantas colunas quantas couberem no orçamento — no máximo `d` (uma por característica original). Não há nenhum limiar de dimensão (tipo "só se $d \leq 3$ "): o próprio `min(d, orcamento)` já reduz gradualmente a quantidade conforme `d` cresce e o orçamento aperta, sem saltos artificiais.

2.4 Termo de raio — e a checagem de redundância

```
self.usa_raio = (squares_dim < d) and (orcamento >= 1)
if self.usa_raio:
    orcamento -= 1; usado += 1
```

O raio ao quadrado, $\sum x_i^2$, resolve em **1 única dimensão** qualquer fronteira radialmente simétrica, em qualquer número de dimensões — é exatamente o que a tarefa `esfera_10d` precisa (sua fronteira é literalmente "raio maior que a mediana").

Por que a condição `squares_dim < d`? Quando todos os `d` quadrados individuais já couberam no orçamento (`squares_dim == d`), o raio deixa de acrescentar qualquer capacidade nova: $\sum x_i^2$ é, por definição, a soma dos `d` quadrados que já estão disponíveis — o Perceptron já pode reconstruí-lo sozinho, dando peso 1 a cada x_i^2 . Incluir o raio nesse caso seria gastar 1 dimensão em uma combinação linear do que já existe. Esta não é uma regra ajustada a uma tarefa: é uma identidade algébrica válida para qualquer `d`.

2.5 Pares cruzados

```
idx_i, idx_j = torch.triu_indices(d, d, offset=1)
n_pares = idx_i.numel()
self.usa_pares = 0 < n_pares <= orcamento
if self.usa_pares:
    self.idx_i, self.idx_j = idx_i, idx_j
    orcamento -= n_pares; usado += n_pares
```

Os termos $x_i x_j$ (para todo par $i < j$) resolvem fronteiras do tipo produto — a fronteira do XOR, por exemplo, é literalmente $x_0 \cdot x_1 = 0$. O número de pares cresce como $C(d, 2) = d(d-1)/2$, ou seja, em $O(d^2)$: para $d=2$ é só 1 par (barato, quase sempre cabe); para $d=10$ já são 45 pares (raramente cabe todo o orçamento). A decisão de incluir ou não **emerge da aritmética do orçamento** (`n_pares <= orcamento`), não de um limiar de dimensão escolhido a dedo.

Por que "tudo ou nada" em vez de incluir parcialmente? Testei uma variante que inclui os pares que couberem (um subconjunto aleatório, em vez de exigir que todos caibam) e ela teve desempenho pior em tarefas de alta dimensão. A explicação: quando os pares não cabem todos, a tarefa provavelmente não depende de estrutura tipo-produto entre pares específicos (como acontece em `esfera_10d`, cuja fronteira é puramente radial) — nesse caso, gastar orçamento em um subconjunto arbitrário de pares apenas rouba espaço das features de Fourier, que generalizam melhor. O corte binário, embora pareça mais "rígido", teve validação empírica a favor (ver Seção 5).

2.6 Random Fourier Features ortogonais (ORF)

O restante do orçamento — pelo menos a reserva mínima, e normalmente bem mais — vai para features de Fourier aleatórias. Esta é a parte "universal" do mapa: aproxima o kernel RBF (gaussiano), que pode representar qualquer fronteira suave, não apenas quadráticas ou tipo-produto.

2.6.1 Ideia teórica (breve)

O teorema de Bochner garante que o kernel RBF pode ser escrito como a esperança de $\cos(w \cdot (x - x'))$ sobre uma distribuição gaussiana de frequências w . Random Fourier Features (Rahimi & Recht, 2007) aproxima essa esperança por uma **média finita**: sorteiam-se `n_rff` vetores de frequência $w_1, \dots, w_{n_rff}$ e um viés de fase b_k , e a característica k é $\cos(w_k \cdot x + b_k)$. O produto interno entre dois pontos nesse espaço transformado aproxima o valor do kernel RBF entre eles — e como o Perceptron encontra um hiperplano nesse espaço, ele efetivamente aproxima um classificador de kernel RBF, capaz de separar fronteiras curvas arbitrárias.

2.6.2 Blocos ortogonais (Orthogonal Random Features)

```
@staticmethod
def _blocos_ortogonais(gen, d, n, device, dtype):
    n_blocos = math.ceil(n / d)
    cols = []
    for _ in range(n_blocos):
        G = torch.randn(d, d, generator=gen, device=device, dtype=dtype)
        Q, _ = torch.linalg.qr(G)
        cols.append(Q)
    U = torch.cat(cols, dim=1)[:n, :n]
    normas = torch.sqrt(torch.distributions.Chi2(d).sample((n,)).to(device=device, dtype=dtype))
    return U * normas
```

Na formulação clássica de RFF, os vetores de frequência são sorteados de forma totalmente independente (gaussiana i.i.d.). Yu et al. (2016, "Orthogonal Random Features") mostraram que forçar esses vetores a serem **mutuamente ortogonais dentro de cada bloco de d direções** reduz a variância da aproximação do kernel, para o mesmo número de features — ou seja, o mesmo orçamento de dimensões aproxima o kernel com mais fidelidade.

A construção: sorteia-se uma matriz gaussiana $d \times d$ e extrai-se sua decomposição QR — as colunas de Q são ortonormais por construção. Como só é possível ter d direções mutuamente ortogonais em um espaço de dimensão d , quando `n_rff > d` repete-se o processo em blocos independentes (cada bloco ortogonal internamente, mas independente dos demais) até cobrir `n_rff` colunas. Por fim, cada coluna é reescalada por uma amostra da distribuição qui (χ) com d graus de liberdade — a mesma distribuição da norma de um vetor gaussiano padrão em R^d — para preservar a distribuição-alvo das frequências, já que a ortogonalização por si só fixa apenas as direções, não as magnitudes.

2.6.3 Calibração da escala pelos dados (pivô por mediana)

```
n_sub = min(300, Xs.shape[0])
sub = Xs[torch.randperm(Xs.shape[0], generator=gen)[:n_sub]]
d2 = torch.cdist(sub, sub) ** 2
sigma0 = torch.sqrt(d2[d2 > 0].median() / 2).clamp_min(1e-3)
gamma0 = 1.0 / sigma0
```

A "largura" do kernel RBF (o quanto duas amostras precisam se afastar para deixarem de ser consideradas parecidas) é calibrada pela **mediana das distâncias entre os próprios pontos de treino** — a mesma heurística usada na referência do professor (`_RFF` no harness). Isso evita usar uma constante fixa "no chute": o pivô se adapta à escala real de cada tarefa, sem depender do rótulo (só de `x_tr`, o que é permitido).

2.6.4 Oitavas multiescala

```
oitavas = 2.0 ** torch.linspace(-2, 2, 5, device=X.device, dtype=X.dtype)
rep = oitavas.repeat(math.ceil(self.n_rff / len(oitavas)))[: self.n_rff]
self.W = W_dir * (gamma0 * rep)
```

Em vez de usar só o pivô calibrado (uma única escala), o orçamento de RFF é dividido em 5 grupos, cada um com uma frequência multiplicada por uma potência de 2 diferente (de 2^{-2} a 2^2 , ou seja, de 1/4 a 4 vezes o pivô). Isso torna o mapa robusto a erros na calibração: mesmo que a mediana não capture exatamente a escala certa para separar uma fronteira mais fina (como os braços entrelaçados da espiral), sempre há um grupo de frequências mais altas cobrindo essa região.

2.6.5 Normalização teórica

```
self._rff_scale = math.sqrt(2.0 / self.n_rff)
```

Fator padrão da formulação de RFF: garante que o produto interno entre dois vetores de características transformados converge, em esperança, para o valor real do kernel RBF — sem esse fator, a soma das características crescería ou encolhería artificialmente conforme o número de features de Fourier mudasse.

3. Passo a passo do `phi(X)`

```
def phi(self, X: torch.Tensor) -> torch.Tensor:
    Xs = (X - self.mu) / self.sd).clamp(-6.0, 6.0)
    partes = [Xs]

    if self.usa_raio:
        partes.append((Xs ** 2).sum(dim=1, keepdim=True))
    if self.squares_dim > 0:
        partes.append(Xs[:, : self.squares_dim] ** 2)
    if self.usa_pares:
        partes.append(Xs[:, self.idx_i] * Xs[:, self.idx_j])

    if self.n_rff > 0:
        Xs64 = Xs.double()
        proj = Xs64 @ self.W.double() + self.b.double()
        partes.append((torch.cos(proj) * self._rff_scale).to(X.dtype))

    return torch.cat(partes, dim=1)
```

3.1 `clamp(-6, 6)` — proteção contra outliers

Antes de qualquer potência ou produto, os valores padronizados são limitados a ± 6 desvios-padrão. Sem isso, um único ponto na cauda do ruído gaussiano (presente em todos os geradores de dados do harness) poderia gerar um valor de x^2 ou de argumento do cosseno muito grande, dominando a atualização de pesos do Perceptron de forma desproporcional a um único ponto.

3.2 Concatenação condicional

Cada bloco só é anexado se o respectivo sinalizador (`usa_raio`, `squares_dim > 0`, `usa_pares`, `n_rff > 0`) estiver ativo — os mesmos calculados em `fit`. A ordem de concatenação é irrelevante para o Perceptron (ele aprende um peso por coluna, independente da posição), mas precisa ser **consistente** entre chamadas, o que é garantido porque esses sinalizadores nunca mudam depois de `fit`.

3.3 Precisão dupla (`.double()`) — por que é necessária

Problema real encontrado durante o desenvolvimento: a checagem de determinismo do harness (`torch.allclose(phi(Xtr[:20]), Ztr[:20])`) por vezes falhava em `float32`. A causa: multiplicação de matriz processada em lote (480+ linhas de uma vez) pode somar os termos da redução numérica em ordem ligeiramente diferente de quando processada em um sub-lote de 20 linhas — gerando uma diferença de ponto flutuante da ordem de 10^{-6} a 10^{-7} . Como o argumento do `cos()` pode ser grande (produto de padronização com frequências de até $4 \times$ o pivô), essa pequena diferença é ampliada o suficiente para estourar a tolerância apertada do `torch.allclose` (10^{-8}). Calcular a projeção e o cosseno em `float64` reduz esse erro numérico por várias ordens de grandeza, eliminando a dependência do tamanho do lote. O resultado final é convertido de volta ao tipo original (`float32`) apenas ao concluir o cálculo.

4. Por que o mapa generaliza (e não decora as 5 tarefas visíveis)

O enunciado avisa que tarefas ocultas da mesma família substituirão as 5 tarefas de teste. A tabela abaixo resume, para cada escolha de projeto, por que ela não depende de nenhuma característica específica de `xor`, `duas_luas`, `circulos`, `espiral` OU `esfera_10d`:

Escolha	Por que generaliza
Reserva mínima de RFF (<code>rem // 4</code>)	Proporção do orçamento restante, não um número fixo — escala com <code>d</code> .
Raio só quando <code>squares_dim < d</code>	Identidade algébrica ($\sum x_i^2$ é combinação linear dos quadrados individuais quando todos cabem) — vale para qualquer <code>d</code> .
Pares cabem por <code>n_pares</code> <code><= orçamento</code>	Consequência direta do crescimento $O(d^2)$ do número de pares — nenhum limiar de dimensão escolhido manualmente.
Pivô de frequência por mediana	Calculado a partir dos dados de treino de <i>cada</i> tarefa individualmente, não de um valor fixo herdado de outra tarefa.
Oitavas multiescala (2^{-2} a 2^2)	Cobertura de frequência simétrica e genérica ao redor do pivô — não foi ajustada observando qual valor "batia melhor" em alguma tarefa.
Semente fixa (<code>manual_seed(0)</code>)	Necessária para satisfazer a exigência de determinismo do enunciado; qualquer inteiro serviria igualmente bem em princípio — não foi escolhida por dar melhor resultado.

5. Código-fonte completo da versão entregue

```
import math
import torch

class Submissao:
    DIM_MAX = 64

    @staticmethod
    def _blocos_ortogonais(gen, d, n, device, dtype):
        """Orthogonal Random Features (Yu et al., 2016): dentro de cada
        bloco de tamanho d, as colunas sao ortogonais entre si; a norma de
        cada coluna eh reamostrada de uma chi(d) para preservar a mesma
        distribuicao-alvo do RFF classico, com menos variancia."""
        n_blocos = math.ceil(n / d)
        cols = []
        for _ in range(n_blocos):
            G = torch.randn(d, d, generator=gen, device=device, dtype=dtype)
            Q, _ = torch.linalg.qr(G)
            cols.append(Q)
        U = torch.cat(cols, dim=1)[:n, :n]
        normas = torch.sqrt(
            torch.distributions.Chi2(d).sample((n,)).to(device=device, dtype=dtype)
        )
        return U * normas

    def fit(self, X: torch.Tensor) -> None:
        self.mu = X.mean(0)
        self.sd = X.std(0) + 1e-8
        d = X.shape[1]
        Xs = (X - self.mu) / self.sd

        rem = self.DIM_MAX - d

        # Reserva minima pra RFF (capacidade "universal", nao especifica de
        # forma quadratica) -- nunca deixamos os termos polinomiais tomarem
        # tudo, senao perdemos a parte que generaliza pra formas nao-
        # quadraticas em qualquer d.
        reserva_rff = max(rem // 4, 4)
        orcamento = max(rem - reserva_rff, 0)
        usado = 0

        # 1) Quadrados individuais: fronteiras elipticas alinhadas aos eixos.
        squares_dim = min(d, orcamento)
        orcamento -= squares_dim; usado += squares_dim
        self.squares_dim = squares_dim

        # 2) Raio^2 (soma dos quadrados): fronteiras radialmente simetricas
        # em qualquer dimensao. So entra quando os quadrados individuais NAO
        # cobrem tudo -- senao eh combinacao linear do que ja existe.
        self.usa_raio = (squares_dim < d) and (orcamento >= 1)
        if self.usa_raio:
            orcamento -= 1; usado += 1

        # 3) Pares cruzados: fronteiras tipo produto/rotacionadas (ex. XOR).
        # Crescem em d^2, entao em d grande simplesmente nao cabem -- eh a
        # aritmetica do orcamento que os desativa, nao um limiar de d.
        idx_i, idx_j = torch.triu_indices(d, d, offset=1)
        n_pares = idx_i.numel()
        self.usa_pares = 0 < n_pares <= orcamento
        if self.usa_pares:
            self.idx_i, self.idx_j = idx_i, idx_j
            orcamento -= n_pares; usado += n_pares

        # 4) Random Fourier Features ortogonais preenchem o resto.
        self.n_rff = max(rem - usado, 0)
        if self.n_rff > 0:
            gen = torch.Generator(device=X.device).manual_seed(0)
            W_dir = self._blocos_ortogonais(gen, d, self.n_rff, X.device, X.dtype)
            self.b = torch.rand(self.n_rff, generator=gen,
                                device=X.device, dtype=X.dtype) * 2 * math.pi

            # Pivo de escala calibrado pela mediana das distancias entre os
            # proprios dados de treino (mesma ideia da referencia do
            # professor) -- nada de constante escolhida a dedo.
            n_sub = min(300, Xs.shape[0])
            sub = Xs[torch.randperm(Xs.shape[0], generator=gen)[:n_sub]]
            d2 = torch.cdist(sub, sub) ** 2
            sigma0 = torch.sqrt(d2[d2 > 0].median() / 2).clamp_min(1e-3)
            gamma0 = 1.0 / sigma0

            # Oitavas geometricas (2^-2 .. 2^2) ao redor do pivo: cobre
            # frequencias grossas e finas mesmo se o pivo nao for exato.
            oitavas = 2.0 ** torch.linspace(-2, 2, 5, device=X.device, dtype=X.dtype)
            rep = oitavas.repeat(math.ceil(self.n_rff / len(oitavas))): self.n_rff]

            self.W = W_dir * (gamma0 * rep)
            self._rff_scale = math.sqrt(2.0 / self.n_rff)

    def phi(self, X: torch.Tensor) -> torch.Tensor:
        # clamp evita que outliers extremos gerem valores enormes de x^2 ou
```

```

# dentro do cos(), causando instabilidade numerica e features
# dominadas por 1 ponto so.
Xs = ((X - self.mu) / self.sd).clamp(-6.0, 6.0)
partes = [Xs]

if self.usa_raio:
    partes.append((Xs ** 2).sum(dim=1, keepdim=True))
if self.squares_dim > 0:
    partes.append(Xs[:, : self.squares_dim] ** 2)
if self.usa_pares:
    partes.append(Xs[:, self.idx_i] * Xs[:, self.idx_j])

if self.n_rff > 0:
    # double: elimina dependencia do tamanho do lote na precisao do
    # cos() -- sem isso, phi(lote inteiro) e phi(sub-lote) podem
    # divergir no ultimo bit e falhar na checagem de determinismo.
    Xs64 = Xs.double()
    proj = Xs64 @ self.W.double() + self.b.double()
    partes.append((torch.cos(proj) * self._rff_scale).to(X.dtype))

return torch.cat(partes, dim=1)

```

6. Referências

- Rahimi, A.; Recht, B. *Random Features for Large-Scale Kernel Machines*. NeurIPS, 2007. (Base teórica das Random Fourier Features usadas para aproximar o kernel RBF.)
- Yu, F. X. et al. *Orthogonal Random Features*. NeurIPS, 2016. (Base teórica da ortogonalização das direções de projeção, usada na Seção 2.6.2.)
- Bochner, S. *Harmonic Analysis and the Theory of Probability*. University of California Press, 1955. (Teorema que garante a representação do kernel RBF como esperança de cossenos, base do método RFF.)

7. Conclusão: o que este desafio ensina sobre deep learning

À primeira vista, o Desafio 1 parece um exercício isolado de engenharia de características para um classificador simples. Mas a tarefa de projetar $\phi(X)$ à mão reproduz, em miniatura, exatamente o problema que motivou a existência do deep learning — e entender essa conexão é o principal valor pedagógico do desafio.

7.1 O Perceptron linear é o "porquê" das camadas profundas

Um Perceptron só separa classes com um hiperplano. Neste desafio, contornamos essa limitação projetando manualmente um espaço onde as classes fossem lineares. Uma rede neural profunda resolve o mesmo problema de um jeito mais geral: em vez de o projetista escolher à mão quais características não-lineares incluir (quadrados, produtos, cossenos), a própria rede **aprende** essas características, camada por camada, ajustando os pesos por gradiente. Cada camada oculta de uma rede é, em essência, uma função ϕ — só que parametrizada e treinável, em vez de fixada de antemão como fizemos aqui.

7.2 Random Features e a primeira camada de uma rede neural

O bloco de Random Fourier Features usado na solução final não é uma analogia solta com redes neurais — é matematicamente o mesmo objeto que a primeira camada de uma rede parcialmente treinada: uma transformação linear ($X @ W + b$) seguida de uma não-linearidade fixa (aqui, $\cos$; em uma rede, tipicamente ReLU ou GELU). A diferença é que, no desafio, a matriz W foi sorteada e nunca é ajustada aos dados; numa rede neural, W começa aleatória (como aqui) e é refinada por retropropagação do erro. Esse paralelo é o mesmo que sustenta, na teoria moderna de deep learning, a análise de redes muito largas via "Neural Tangent Kernel" — que também trata os pesos da rede como essencialmente aleatórios em torno da inicialização.

7.3 A tensão entre viés indutivo e flexibilidade

A comparação entre manter termos estruturados (quadrados, pares cruzados) e usar apenas características aleatórias genéricas reproduz, em escala pequena, uma das decisões mais centrais no projeto de arquiteturas de deep learning: quanto de conhecimento prévio sobre o problema (viés indutivo) vale a pena embutir na estrutura do modelo, em vez de deixar que ele aprenda tudo do zero. Redes convolucionais embutem a suposição de que padrões locais e translações importam; transformers embutem a suposição de que relações entre posições distantes de uma sequência importam. Da mesma forma, embutir $x \cdot y$ explicitamente no mapa deste desafio parte da suposição de que fronteiras tipo-produto (como a do XOR) são comuns o bastante para merecer uma característica dedicada — e essa suposição, quando correta, faz o modelo aprender com muito menos dados e menos iterações de treino do que se tivesse de descobrir a mesma estrutura por conta própria.

7.4 Estabilidade numérica não é detalhe de implementação

O problema de determinismo resolvido com `float64` no cálculo do cosseno ilustra outra lição central do treinamento de redes profundas: instabilidades numéricas pequenas (erros de arredondamento, gradientes muito grandes ou muito pequenos) podem se propagar e comprometer todo um pipeline de treinamento. É exatamente por isso que redes profundas reais dependem de técnicas como normalização de camadas (BatchNorm, LayerNorm), inicialização cuidadosa de pesos e, por vezes, treinamento em precisão mista — soluções conceitualmente aparentadas ao `clamp` e à normalização por mediana usados nesta solução.

7.5 Da engenharia manual à representação aprendida

Historicamente, a evolução do aprendizado de máquina seguiu quase esse mesmo caminho: métodos de kernel e engenharia manual de características dominaram até meados dos anos 2010, quando redes profundas passaram a aprender representações diretamente dos dados brutos, superando o desempenho de características desenhadas à mão em praticamente todo domínio com dados suficientes. Fazer esse desafio "no braço" — decidindo à mão quais características criar e validando cada decisão com experimentos controlados — é uma forma direta de sentir, na prática, o problema que o deep learning resolve de maneira automática: aprender a própria representação dos dados, em vez de projetá-la.